

Taller 2: sistema de batallas Pokémon

Saul Leonardo Cruz Romero¹, Isabella Hermosa Losada¹,
Alejandro Macias Barrios¹

¹Pontificia Universidad Javeriana, Bogotá D. C., Colombia.

1. Descripción del problema

1.1. Investigación sobre el sistema de batallas Pokémon

Los Pokémon son criaturas de diferentes formas y tamaños que pueden vivir en estado salvaje o junto a los seres humanos [1]. En este contexto, una persona puede convertirse en entrenador y organizar un equipo para participar en enfrentamientos. Cada equipo puede contener como máximo seis Pokémon [2], cuyo orden puede cambiarse para aprovechar sus características y relaciones elementales durante una batalla.

Al comenzar un enfrentamiento, cada entrenador utiliza primero el Pokémon que encabeza su equipo. Esta organización permite saber cuál entra inicialmente al campo y cuál será reemplazado si queda debilitado.

1.2. Acciones y prioridad durante la batalla

Cuando comienza el combate aparecen cuatro acciones principales: Luchar, Mochila, Pokémon y Correr. Las acciones relacionadas con objetos, cambios o retirada tienen prioridad. Si ambos entrenadores eligen una acción con la misma prioridad, se considera quién inició el enfrentamiento. Al escoger Luchar, el orden de ataque depende principalmente de las estadísticas de los Pokémon.

1.3. Estadísticas de combate

La primera estadística que se compara para decidir quién ataca es la velocidad. Normalmente, el Pokémon con el valor más alto actúa primero, aunque existen estados, condiciones del clima u otras interacciones capaces de modificar este orden.

El Ataque y el Ataque Especial influyen en la fuerza de los movimientos. El Ataque se utiliza en movimientos físicos o de contacto, mientras que el Ataque Especial representa la fuerza de los movimientos que liberan energía o se realizan a distancia



Figura 1 Organización de un equipo Pokémon y selección del primer integrante.



Figura 2 Acciones principales disponibles durante un combate Pokémon.

[3]. De forma similar, la Defensa y la Defensa Especial determinan la resistencia frente a movimientos físicos y especiales.

1.4. Relaciones entre tipos

Para calcular el daño también deben compararse el tipo del movimiento y el tipo del Pokémon que recibe el ataque. La relación puede aumentar, reducir o anular el daño, por lo que elegir el movimiento adecuado puede cambiar el resultado del enfrentamiento.

- **Efectivo:** el movimiento causa el doble de daño a los tipos indicados.
- **No efectivo:** el movimiento causa la mitad del daño a los tipos indicados.
- **Débil contra:** el Pokémon recibe el doble de daño de los tipos indicados.
- **Inmune:** el Pokémon no recibe daño de los tipos indicados.



Figura 3 Estadísticas de un Pokémon utilizadas durante el combate.

TABLA DE TIPOS BASICA	EFFECTIVO	NO EFFECTIVO	DÉBIL CONTRA	INMUNE
ACERO				
VOLADOR				
AGUA				
HIELO				
PLANTA				
BICHO				
ELECTRICO				
NORMAL				
ROCA				
TIERRA				
FUEGO				
LUCHA				
HADA				
PSIQUICO				
VENENO				
DRAGÓN				
FANTASMA				
SINIESTRO				

Figura 4 Tabla básica de relaciones entre tipos de Pokémon.

Por ejemplo, un movimiento de tipo Fuego es efectivo contra Acero, Hielo, Planta y Bicho, pero resulta poco efectivo contra Agua, Roca, Fuego y Dragón. A su vez, un Pokémon de tipo Fuego es débil frente a movimientos de Agua, Roca y Tierra. Conocer estas relaciones ayuda a maximizar el daño causado y reducir el daño recibido.

Como se muestra en la presente gráfica, los atacantes pueden llegar a tener un multiplicador en el daño que realizan dependiendo del elemento del defensor. En dado caso que no se muestre un multiplicador entre el atacante y defensor, como, por

ejemplo: fuego tiene x2 a planta, pero x0.5 a agua, este contara implícitamente como un multiplicador x1 al daño que se realizará.

La relación va de atacante a defensor ya que, el atacante es el que realiza el movimiento de algún elemento independiente del elemento natural del Pokemon, mientras que el defensor se encarga de recibir ese ataque comparándolo con el elemento natural del pokemon para calcular el multiplicador de daño que será asignado al movimiento realizado. En caso de querer invertir estos roles no tendría sentido ya que, rompe el sentido natural de causa y efecto, véase como causa, la acción principal o disparador del evento, el ataque y como efecto el multiplicador que se le proveerá al movimiento dependiendo del elemento natural que tenga el defensor.

Ademas, para realizar los cálculos de los multiplicadores de daño y defensa se parte desde una base, la cual es el ataque, en caso de querer partir desde la defensa, colapsaría este sistema, gracias a que, no tiene una base la cual regirse para aplicar la matemática necesaria para continuar.

Los valores retornados por las funciones influyen en la función “calculateDamage()” debido a que, en caso de que sea un ataque eficaz, tendrá una bonificación de x2 en el daño a realizar, mientras que, si es un ataque no muy eficaz, tendrá un multiplicador de x0.5 al daño a realizar. Esto afecta en que tantos HP se le van a restar al Pokemon oponente después de recibir el ataque.

1.5. Condición de finalización

Los puntos de salud o HP indican cuánto daño puede resistir un Pokémon. Cuando llegan a cero, el Pokémon queda debilitado y sale del combate. Si todos los integrantes de un equipo quedan debilitados, su entrenador pierde la batalla. Además, los aumentos o disminuciones temporales de estadísticas deben conservarse en el historial para representar correctamente la evolución del enfrentamiento.

2. Componente 1: Organización de la información

Este componente organiza la información leída desde `trainers.txt` y `operations.txt` para cargar entrenadores, ejecutar batallas, comparar equipos e imprimir los datos almacenados. Las instrucciones permiten utilizar el simulador sin intervenir directamente en sus estructuras internas.

2.1. Descripción de los comandos

BATTLE

Recibe los nombres de dos entrenadores y de los equipos que se enfrentarán. La instrucción solo se ejecuta si los cuatro nombres están completos y existen en el simulador. Al finalizar, guarda la batalla en el historial y muestra el desarrollo del enfrentamiento; si falta información o algún participante no es válido, informa el error y detiene la lectura de operaciones.

COMPARE

Recibe los nombres de dos entrenadores y enfrenta todos los equipos del primero con todos los del segundo. Para funcionar, ambos nombres deben estar presentes y la comparación debe poder realizarse. Como resultado, almacena las batallas y estadísticas, imprime los resultados generales y muestra los equipos con mayor porcentaje de victorias de cada entrenador.

PRINT

Muestra los entrenadores, equipos y Pokémon que ya se encuentran almacenados en el simulador. Esta instrucción no necesita información adicional después de su nombre y no modifica los datos guardados.

2.2. Diseño

El análisis se divide en dos elementos. Primero se presenta la tabla de las funciones auxiliares definidas en `main.cxx`. Después se muestra el diagrama de flujo de la función `main`. De acuerdo con la indicación del profesor, `main` no se repite en la tabla porque se representa mediante el diagrama.

Tabla de funciones definidas en `main.cxx`

Cuadro 1: Funciones definidas en `main.cxx`.

Función Nombre de la función	Propósito Descripción breve	Datos de entrada Parámetros o infor- mación procesada	Resultado Valor retornado o efecto producido	Funciones que utiliza Funciones relevantes que invoca
<code>stringToPokemonType</code>	Convierte el tipo elemental leído como texto en el valor correspondiente de <code>PokemonType</code> .	<code>typeName (conststd::string&)</code> : texto con el tipo. La referencia evita copiarlo y <code>const</code> impide modificarlo.	Retorna el <code>PokemonType</code> correspondiente. Si no coincide con los tipos anteriores, retorna <code>FLYING</code> . No modifica el simulador.	No invoca otras funciones; utiliza condiciones para comparar el texto con los tipos disponibles.
<code>loadTeam</code>	Lee los Pokémon de <code>trainers.txt</code> y los agrega al equipo hasta encontrar <code>END_TEAM</code> .	<code>inputFile (std::ifstream&)</code> : continúa leyendo el archivo. <code>team (Team&)</code> : permite modificar directamente el equipo original.	Agrega Pokémon. Retorna <code>true</code> al encontrar <code>END_TEAM</code> ; retorna <code>false</code> por datos, instrucciones, inserciones o cierre incorrectos.	<code>getline</code> , <code>stringToPokemonType</code> , <code>getPokemonCount</code> , <code>addPokemon</code> y <code>getName</code> .
<code>loadTrainer</code>	Lee los equipos de <code>trainers.txt</code> y los agrega al entrenador hasta encontrar <code>END_TRAINER</code> .	<code>inputFile (std::ifstream&)</code> : conserva la lectura. <code>trainer (Trainer&)</code> : permite modificar directamente el entrenador original.	Agrega equipos. Retorna <code>true</code> al encontrar <code>END_TRAINER</code> ; retorna <code>false</code> por nombres, instrucciones, equipos o cierre incorrectos.	<code>getline</code> , <code>loadTeam</code> , <code>addTeam</code> y <code>getName</code> .
<code>loadTrainers</code>	Carga los entrenadores de <code>trainers.txt</code> con sus equipos y los almacena dentro del simulador.	<code>fileName (conststd::string&)</code> : identifica el archivo. <code>simulator (BattleSimulator&)</code> : recibe los entrenadores cargados.	Modifica el simulador agregando entrenadores. Retorna <code>false</code> por apertura fallida, formato incorrecto, nombres faltantes o inserciones fallidas.	<code>is_open</code> , <code>getline</code> , <code>loadTrainer</code> , <code>addTrainer</code> y <code>close</code> .

Continúa en la página siguiente.

Cuadro 1 (continuación)

Función Nombre de la función	Propósito Descripción breve	Datos de entrada Parámetros o infor- mación procesada	Resultado Valor retornado o efecto producido	Funciones que utiliza Funciones relevantes que invoca
<code>executeBattleInstru- ction</code>	Extrae cuatro nombres de BATTLE, ejecuta la batalla y muestra el historial del enfrentamiento.	<code>lineStream (std::ist- ringstream&)</code> : contiene cuatro nombres. <code>simul- ator (BattleSimulato- r&)</code> : ejecuta y almacena la batalla.	Agrega una batalla al historial. Retorna <code>true</code> al ejecutarla; retorna <code>false</code> si faltan nombres o no puede realizarse.	<code>executeBattle</code> , <code>getBat- tleCount</code> , <code>getBattle</code> y <code>printBattleHistory</code> .
<code>printBestTeams</code>	Consulta los equipos con mayor porcentaje de victorias de un entrenador y muestra sus nombres.	<code>simulator (constBattl- eSimulator&)</code> : contiene los resultados. <code>train- erName (conststd::s- tring&)</code> : identifica al entrenador consultado.	Imprime los mejores equipos. No retorna valores, no modifica el simulador ni detecta errores directamente.	<code>getBestTeams</code> , <code>begin</code> y <code>end</code> .
<code>executeComparisonInst- ruction</code>	Extrae dos entrenadores de COMPARE, compara todos sus equipos y muestra los resultados.	<code>lineStream (std::ist- ringstream&)</code> : contiene dos nombres. <code>simulat- or (BattleSimulator- &)</code> : realiza y almacena la comparación.	Modifica los resul- tados y el historial. Retorna <code>true</code> al com- parar; retorna <code>false</code> si faltan nombres o la comparación falla.	<code>compareTrainers</code> , <code>prin- tComparisonResults</code> y <code>printBestTeams</code> .
<code>executePrintInstructi- on</code>	Muestra los entrenadores, equipos y Pokémon almacenados actualmente en el simulador.	<code>lineStream (std::istr- ingstream&)</code> : se recibe, pero no se utiliza. <code>si- mulator (BattleSimu- lator&)</code> : contiene los entrenadores.	Imprime la informa- ción y retorna <code>true</code> . No modifica el simula- dor ni detecta errores directamente.	<code>printTrainers</code> .

Continúa en la página siguiente.

Cuadro 1 (continuación)

Función Nombre de la función	Propósito Descripción breve	Datos de entrada Parámetros o infor- mación procesada	Resultado Valor retornado o efecto producido	Funciones que utiliza Funciones relevantes que invoca
<code>executeOperations</code>	Lee <code>operations.txt</code> , identifica cada instruc- ción y envía sus datos a la función correspondiente.	<code>fileName</code> (<code>conststd: :string&</code>): identifica el archivo. <code>simulator (BattleSimulator&)</code> : conserva los cambios producidos.	Puede modificar el si- mulador. Retorna <code>false</code> si el archivo no abre, la instrucción es descono- cida o alguna operación falla.	<code>getline</code> , <code>executeBattl eInstruction</code> , <code>execute ComparisonInstruction</code> y <code>executePrintInstruc tion</code> .
<code>printExecutedBattles</code>	Recorre el historial y muestra los participantes, equipos y ganador de cada batalla ejecutada.	<code>simulator</code> (<code>constB attleSimulator&</code>): contiene el historial, consultado sin copiarlo ni modificarlo.	Imprime la cantidad y el resumen de las batallas. No retorna valores ni modifica el simulador.	<code>getBattleCount</code> , <code>get Battle</code> y funciones de consulta de <code>Battle</code> y <code>Te am</code> .

Diagrama de flujo de la función main

El flujo comienza con la creación del simulador. Luego se cargan los entrenadores desde `trainers.txt`; si la carga falla, el programa retorna 1. Cuando la carga funciona, se procesan las instrucciones de `operations.txt` y se imprime el resumen de las batallas que alcanzaron a ejecutarse. Finalmente, el programa retorna 0 si todas las operaciones terminan correctamente o 1 si alguna falla.

Diagrama de flujo de la función int main

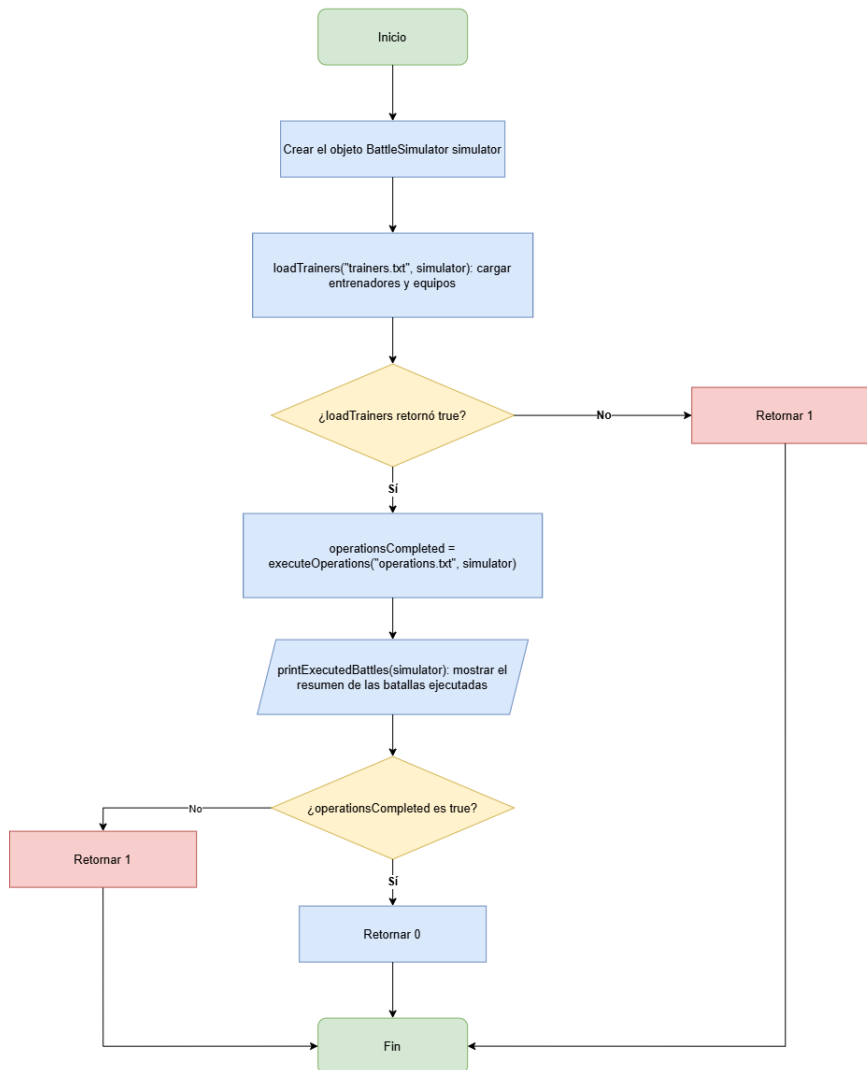
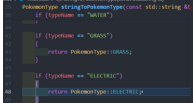
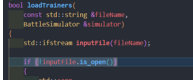


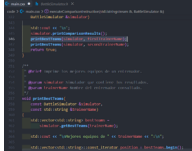
Figura 5 Diagrama de flujo de la función main.

3. Corrección de errores y advertencias

La revisión se realizó compilando los archivos fuente de forma incremental con C++17. Después de cada cambio se repitió la compilación para comprobar que el problema desapareciera y determinar si el compilador revelaba un error posterior. Una vez corregidos los cinco errores de compilación, se generó el ejecutable y se procesaron los archivos de entrada para identificar un error lógico durante la ejecución.

Comandos de verificación: `g++-std=c++17-c*.cxx; g++-std=c++17-otaller2_punto3*.o; ./taller2_punto3`. La compilación y el enlace finales no mostraron errores ni advertencias.

Tipo de error	Descripción del error	Causa probable del error	Solución aplicada	Resultado tras corrección
Compilación	La conversión del texto <code>ELECTRIC</code> utilizaba un identificador que no existe en la enumeración <code>PokemonType</code> .	En <code>stringToPokemonType</code> se escribió <code>PokemonType::Electric</code> , pero <code>PokemonType</code> declara <code>ELECTRIC</code> en mayúsculas. C++ diferencia mayúsculas y minúsculas. 	Se reemplazó <code>PokemonType::Electric</code> por <code>PokemonType::ELECTRIC</code> , sin modificar la enumeración ni la firma de la función. 	El error desapareció en la siguiente compilación. El compilador continuó informando el problema de <code>maximumHealth</code> .
Compilación	La lectura de los datos de un Pokémon empleaba una variable que no había sido declarada.	La variable fue declarada como <code>maximumHealth</code> , pero en la extracción se escribió <code>maximumHealth</code> , omitiendo la letra <code>h</code> final. 	Se corrigió el identificador de la extracción a <code>maximumHealth</code> para que coincidiera con la declaración existente. 	El error dejó de aparecer. La compilación siguiente mostró los problemas de <code>inputFile</code> , <code>printComparisonResults</code> y <code>printBestTeams</code> .
Compilación	La validación de apertura del archivo de entrenadores empleaba un identificador no declarado.	El objeto se llama <code>inputFile</code> , pero la condición utilizaba <code>InputFile</code> . Ambos nombres son diferentes para el compilador. 	Se sustituyó <code>InputFile</code> por <code>inputFile</code> , conservando la condición que detecta cuando el archivo no está abierto. 	El error fue eliminado. En la recompilación permanecieron únicamente <code>printComparisonResults</code> y <code>printBestTeams</code> .

Tipo de error	Descripción del error	Causa probable del error	Solución aplicada	Resultado tras corrección
Compilación	Se intentaba invocar un método inexistente de <code>BattleSimulator</code> debido a una diferencia de capitalización.	La interfaz declara <code>printComparisonResults</code> , pero <code>executeComparisonInstruction</code> llamaba <code>PrintComparisonResults</code> con la letra inicial en mayúscula. 	Se cambió la llamada a <code>simulator.printComparisonResults()</code> , respetando el nombre definido en <code>BattleSimulator.h</code> . 	La llamada al método fue reconocida. La compilación posterior informó solamente el problema de <code>printBestTeams</code> .
Compilación	La función <code>printBestTeams</code> era utilizada antes de que el compilador conociera su declaración.	Su definición completa aparecía después de <code>executeComparisonInstruction</code> y no existía una declaración previa antes del llamado. 	Se trasladó la definición completa de <code>printBestTeams</code> antes de <code>executeComparisonInstruction</code> , conservando su firma y su cuerpo. 	La compilación y el enlace se completaron sin mensajes de error ni advertencia mediante los comandos indicados.

Tipo de error	Descripción del error	Causa probable del error	Solución aplicada	Resultado tras corrección
Ejecución	El programa informaba que <code>operations.txt</code> no podía abrirse, aunque el archivo estaba presente en el directorio de ejecución.	<code>executeOperations</code> evaluaba <code>if(inputFile.is_open())</code> y entraba al bloque de error precisamente cuando la apertura había sido exitosa. 	Se añadió el operador de negación: <code>if(!inputFile.is_open())</code> . El error se informa únicamente cuando el archivo no está abierto. 	<code>operations.txt</code> fue procesado y PRINT mostró los dos entrenadores registrados. Después apareció un fallo distinto, relacionado con TODO#03 y TODO#08 todavía pendientes. 

Tipo de error	Descripción del error	Causa probable del error	Solución aplicada	Resultado tras corrección
Ejecución	Después de procesar la primera instrucción BATTLE , el programa intentaba consultar la última batalla del historial y terminaba abruptamente porque no existía ninguna batalla almacenada.	<code>BattleSimulator::executeBattle()</code> todavía conservaba la implementación provisional del <code>TODO#08</code> : retornaba <code>true</code> sin construir, ejecutar ni almacenar una batalla. Debido a ese resultado, <code>executeBattleInstruction()</code> continuaba con <code>getBattleCount()-1</code> . Como la cantidad era cero, la resta producía una posición inválida que posteriormente se enviaba a <code>getBattle()</code> .	Se implementó el <code>TODO#08</code> para buscar ambos entrenadores, comprobar la existencia de los equipos, construir una batalla con las copias correspondientes, ejecutarla mediante <code>runBattle()</code> y agregarla al final de <code>battleHistory</code> . La función retorna <code>true</code> solamente después de ejecutar y almacenar la batalla.	La instrucción BATTLE pudo consultar e imprimir la batalla recién almacenada sin acceder a una posición inexistente. Después se procesaron las demás instrucciones de <code>operations.txt</code> , el programa finalizó con código 0 y el historial reportó ocho batallas ejecutadas. No volvió a presentarse el fallo de acceso al <code>deque</code> .

4. Tipos Abstractos de Datos (TAD)

4.1. TAD Pokemon

Representa un Pokémon del simulador y conserva su tipo, salud y estadísticas utilizadas durante las batallas.

Estado

- **Nombre (name), cadena de caracteres.** Identifica al Pokémon.
- **Tipo (type), tipo elemental.** Determina las fortalezas y debilidades del Pokémon frente a otros tipos.
 - *Condiciones de la variable:* Solo puede tomar los valores FIRE, WATER, GRASS, ELECTRIC, GROUND o FLYING.
- **Salud máxima (maximumHealth), numérico entero.** Indica la cantidad total de puntos de salud del Pokémon.
 - *Condiciones de la variable:* Es mayor que cero. Conserva el valor establecido al crear el Pokémon.
- **Salud actual (currentHealth), numérico entero.** Indica los puntos de salud que conserva el Pokémon.
 - *Condiciones de la variable:* Se encuentra entre cero y la salud máxima. Comienza con el mismo valor de la salud máxima. No disminuye por debajo de cero.
- **Ataque (attack), numérico entero.** Representa la capacidad ofensiva utilizada para calcular el daño.
 - *Condiciones de la variable:* Es un valor no negativo. Conserva el valor establecido al crear el Pokémon.
- **Defensa (defense), numérico entero.** Representa la capacidad utilizada para disminuir el daño recibido.
 - *Condiciones de la variable:* Es un valor no negativo. Conserva el valor establecido al crear el Pokémon.
- **Velocidad (speed), numérico entero.** Determina cuál Pokémon ataca primero.
 - *Condiciones de la variable:* Es un valor no negativo. Conserva el valor establecido al crear el Pokémon.

Interfaz

- **crearPokemon(name, type, maximumHealth, attack, defense, speed): Pokemon.** Inicia un Pokémon con su identificación, tipo elemental y estadísticas de batalla.
 - *Precondiciones:* maximumHealth es mayor que cero. attack es un valor no negativo. defense es un valor no negativo. speed es un valor no negativo.

- *Poscondiciones:* Se obtiene un Pokémon con el nombre, tipo y estadísticas recibidos. La salud actual queda igual a la salud máxima.
- **getName(): cadena de caracteres.** Permite conocer el nombre del Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre almacenado. Los datos del Pokémon conservan sus valores.
- **getType(): tipo elemental.** Permite conocer el tipo elemental del Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el tipo almacenado. Los datos del Pokémon conservan sus valores.
- **getMaximumHealth(): numérico entero.** Permite conocer la cantidad máxima de puntos de salud del Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la salud máxima. Los datos del Pokémon conservan sus valores.
- **getCurrentHealth(): numérico entero.** Permite conocer la cantidad de puntos de salud que conserva el Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la salud actual. Los datos del Pokémon conservan sus valores.
- **getAttack(): numérico entero.** Permite conocer el valor de ataque utilizado para calcular el daño causado.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el valor de ataque. Los datos del Pokémon conservan sus valores.
- **getDefense(): numérico entero.** Permite conocer el valor de defensa utilizado para disminuir el daño recibido.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el valor de defensa. Los datos del Pokémon conservan sus valores.
- **getSpeed(): numérico entero.** Permite conocer la velocidad utilizada para establecer el orden de ataque.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el valor de velocidad. Los datos del Pokémon conservan sus valores.
- **receiveDamage(damage): vacío.** Disminuye la salud actual del Pokémon según la cantidad de daño recibida.

- *Precondiciones:* **damage** es un valor no negativo.
 - *Poscondiciones:* La salud actual disminuye en la cantidad indicada. La salud actual queda en cero cuando el daño es mayor que la salud disponible. El nombre, el tipo, la salud máxima, el ataque, la defensa y la velocidad conservan sus valores.
- **restoreHealth(): vacío.** Recupera completamente la salud del Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* La salud actual queda igual a la salud máxima. Las demás características y estadísticas conservan sus valores.
- **isFainted(): booleano.** Determina si el Pokémon se quedó sin puntos de salud.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando la salud actual es cero. Retorna falso cuando la salud actual es mayor que cero. Los datos del Pokémon conservan sus valores.
- **print(): vacío.** Presenta una tarjeta con el nombre, tipo, salud y estadísticas del Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se muestra el nombre y el tipo elemental. Se muestran la salud actual y la salud máxima. Se muestran el ataque, la defensa y la velocidad. Los datos del Pokémon conservan sus valores.

4.2. TAD Team

Representa un equipo Pokémon identificado por un nombre y formado por una secuencia ordenada de máximo seis integrantes. El orden determina cuál Pokémon aparece primero y estará disponible antes durante una batalla.

Estado

- **Capacidad máxima (maxPokemon), numérico entero.** Indica la mayor cantidad de Pokémon que puede contener un equipo.
- *Condiciones de la variable:* Su valor es seis. Es compartida por todos los equipos. No cambia durante el funcionamiento del simulador.
- **Nombre (name), cadena de caracteres.** Identifica al equipo.
- **Pokémon del equipo (pokemonList), secuencia ordenada de Pokémon.** Almacena los integrantes del equipo.
- *Condiciones de la variable:* Puede contener entre cero y seis Pokémon. No contiene dos Pokémon con el mismo nombre. La posición de cada Pokémon determina su orden dentro del equipo.

Interfaz

- **crearTeam(name): Team.** Inicia un equipo Pokémon con el nombre recibido y sin integrantes.

- *Precondiciones:* Ninguna.
- *Poscondiciones:* Se obtiene un equipo con el nombre recibido. La cantidad de Pokémon queda en cero.
- **getName(): cadena de caracteres.** Permite conocer el nombre del equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre almacenado. Los datos del equipo conservan sus valores.
- **getPokemonCount(): numérico entero.** Informa cuántos Pokémon pertenecen actualmente al equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la cantidad de Pokémon almacenados. Los datos del equipo conservan sus valores.
- **isEmpty(): booleano.** Determina si el equipo no tiene Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando la cantidad de Pokémon es cero. Retorna falso cuando el equipo contiene al menos un Pokémon. Los datos del equipo conservan sus valores.
- **isFull(): booleano.** Determina si el equipo alcanzó su capacidad máxima de seis Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando el equipo contiene seis Pokémon. Retorna falso cuando contiene menos de seis. Los datos del equipo conservan sus valores.
- **getNotFoundPosition(): posición de Pokemon.** Retorna la posición especial utilizada para indicar que no se encontró un Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna una posición que no corresponde a ningún Pokémon del equipo. No modifica los datos del equipo.
- **findPokemon(pokemonName): posición de Pokemon.** Busca un Pokémon mediante su nombre.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si encuentra un Pokémon con el nombre recibido, retorna su posición. Si no lo encuentra, retorna la posición especial proporcionada por **getNotFoundPosition()**. No modifica los integrantes ni el orden del equipo.
- **containsPokemon(pokemonName): booleano.** Comprueba si el equipo contiene un Pokémon con el nombre recibido.
 - *Precondiciones:* Ninguna.

- *Poscondiciones:* Retorna verdadero cuando el Pokémon se encuentra en el equipo. Retorna falso cuando el Pokémon no se encuentra en el equipo. Los datos del equipo conservan sus valores.
- **addPokemon(pokemon, position): booleano.** Intenta agregar una copia del Pokémon recibido en la posición indicada.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando el equipo no está lleno, el nombre no está repetido y **position** se encuentra entre cero y la cantidad actual de Pokémon. Cuando retorna verdadero, la cantidad de integrantes aumenta en uno. Cuando retorna verdadero, el nuevo Pokémon queda ubicado en **position**. Los Pokémon que ya estaban en el equipo mantienen el mismo orden entre ellos. Retorna falso cuando el equipo está lleno. Retorna falso cuando ya existe un Pokémon con el mismo nombre. Retorna falso cuando **position** supera la cantidad actual de Pokémon. Cuando retorna falso, los datos y el orden del equipo conservan sus valores.
- **removePokemon(pokemonName): booleano.** Intenta eliminar del equipo al Pokémon que tenga el nombre recibido.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si el Pokémon existe, es eliminado y la operación retorna verdadero. Cuando retorna verdadero, la cantidad de integrantes disminuye en uno. Los Pokémon restantes mantienen el mismo orden entre ellos. Si el Pokémon no existe, retorna falso. Cuando retorna falso, los datos y el orden del equipo conservan sus valores.
- **changePokemonPosition(pokemonName, newPosition): booleano.** Intenta trasladar al Pokémon indicado a otra posición dentro del equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando el Pokémon existe y **newPosition** corresponde a una posición ocupada del equipo. Cuando retorna verdadero, el Pokémon encontrado queda ubicado en **newPosition**. Cuando retorna verdadero, los demás Pokémon se desplazan según corresponda. La cantidad de integrantes conserva su valor. Retorna falso cuando el Pokémon no existe. Retorna falso cuando **newPosition** no corresponde a una posición ocupada. Cuando retorna falso, el orden del equipo conserva su valor.
- **restoreTeam(): vacío.** Recupera completamente la salud de todos los Pokémon del equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* La salud actual de cada Pokémon queda igual a su salud máxima. El nombre del equipo conserva su valor. La cantidad y el orden de los Pokémon conservan sus valores. Los nombres, tipos y estadísticas de los Pokémon conservan sus valores.

- **isDefeated(): boolean.** Determina si el equipo no tiene ningún Pokémon disponible para continuar una batalla.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando el equipo está vacío. Retorna verdadero cuando todos sus Pokémon están debilitados. Retorna falso cuando al menos un Pokémon conserva puntos de salud. Los datos del equipo conservan sus valores.
- **getFirstAvailablePokemon(): posición de Pokemon.** Busca el primer Pokémon del equipo que todavía puede combatir.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si existe al menos un Pokémon que no está debilitado, retorna la posición del primero según el orden del equipo. Si el equipo está vacío o todos sus Pokémon están debilitados, retorna la posición especial proporcionada por `getNotFoundPosition()`. No modifica la salud ni el orden de los Pokémon.
- **print(): vacío.** Presenta el nombre del equipo, la cantidad de integrantes y la información de sus Pokémon.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se muestra el nombre del equipo. Se muestra la cantidad de Pokémon respecto al límite de seis. Si el equipo está vacío, se informa que no contiene Pokémon. Si contiene Pokémon, se muestran su posición y su tarjeta comenzando por el primero. Los datos y el orden del equipo conservan sus valores.

4.3. TAD Trainer

Representa un entrenador Pokémon identificado por un nombre y encargado de una secuencia ordenada de equipos. Permite registrar, buscar, eliminar, consultar y restaurar sus equipos.

Estado

- **Nombre (name), cadena de caracteres.** Identifica al entrenador.
- **Equipos (teamList), secuencia ordenada de equipos Pokémon.** Almacena los equipos pertenecientes al entrenador.
 - *Condiciones de la variable:* Puede contener cero o más equipos. No contiene dos equipos con el mismo nombre.

Interfaz

- **crearTrainer(name): Trainer.** Inicia un entrenador con el nombre recibido y sin equipos registrados.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se obtiene un entrenador con el nombre recibido. La cantidad de equipos queda en cero.
- **getName(): cadena de caracteres.** Permite conocer el nombre del entrenador.
 - *Precondiciones:* Ninguna.

- *Poscondiciones:* Retorna el nombre almacenado. Los datos del entrenador conservan sus valores.
- **getTeamCount(): numérico entero.** Informa cuántos equipos pertenecen actualmente al entrenador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la cantidad de equipos almacenados. Los datos del entrenador conservan sus valores.
- **isEmpty(): booleano.** Determina si el entrenador no tiene equipos registrados.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando la cantidad de equipos es cero. Retorna falso cuando el entrenador tiene al menos un equipo. Los datos del entrenador conservan sus valores.
- **findTeam(teamName): numérico entero.** Busca un equipo por su nombre y retorna la posición que ocupa dentro del entrenador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la posición del equipo cuando encuentra el nombre recibido. Retorna -1 cuando el equipo no pertenece al entrenador. Los equipos conservan sus datos y su orden.
- **containsTeam(teamName): booleano.** Comprueba si el entrenador tiene un equipo con el nombre recibido.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando el equipo pertenece al entrenador. Retorna falso cuando el equipo no pertenece al entrenador. Los datos del entrenador conservan sus valores.
- **addTeam(team): booleano.** Agrega una copia del equipo recibido cuando su nombre no se encuentra registrado.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si no existe otro equipo con el mismo nombre, se agrega una copia al final y retorna verdadero. Cuando retorna verdadero, la cantidad de equipos aumenta en uno. Los equipos que ya estaban registrados mantienen el mismo orden entre ellos. Si el nombre ya existe, retorna falso. Cuando retorna falso, los datos y el orden de los equipos conservan sus valores.
- **removeTeam(teamName): booleano.** Intenta eliminar del entrenador al equipo que tenga el nombre recibido.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si el equipo existe, es eliminado y la operación retorna verdadero. Cuando retorna verdadero, la cantidad de equipos disminuye en uno. Los

equipos ubicados después del eliminado avanzan una posición. Los equipos restantes mantienen el mismo orden entre ellos. Si el equipo no existe, retorna falso. Cuando retorna falso, los datos y el orden de los equipos conservan sus valores.

- **getTeam(position): Team.** Retorna el equipo almacenado en la posición indicada.
 - *Precondiciones:* **position** es mayor o igual que cero. **position** es menor que la cantidad de equipos del entrenador.
 - *Poscondiciones:* Retorna el equipo ubicado en la posición recibida. No modifica la cantidad ni el orden de los equipos.
- **restoreAllTeams(): vacío.** Recupera completamente la salud de todos los Pokémon pertenecientes al entrenador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* La salud actual de cada Pokémon queda igual a su salud máxima. La cantidad y el orden de los equipos conservan sus valores. La cantidad y el orden de los Pokémon conservan sus valores. Los nombres, tipos y estadísticas de los Pokémon conservan sus valores.
- **print(): vacío.** Presenta el nombre del entrenador, la cantidad de equipos y la información de cada uno.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se muestra el nombre del entrenador. Se muestra la cantidad de equipos registrados. Si no existen equipos, se informa que el entrenador no tiene equipos registrados. Si existen equipos, se muestra la posición y la información de cada uno comenzando por el primero. Los datos y el orden de los equipos conservan sus valores.

4.4. TAD Battle

Representa una batalla entre dos equipos Pokémon pertenecientes a dos entrenadores. Utiliza copias de los equipos para que los cambios de salud producidos durante el enfrentamiento no afecten a los equipos originales.

Estado

- **Nombre del primer entrenador (firstTrainerName), cadena de caracteres.** Identifica al propietario del primer equipo.
- **Nombre del segundo entrenador (secondTrainerName), cadena de caracteres.** Identifica al propietario del segundo equipo.
- **Primer equipo (firstTeam), equipo Pokémon.** Conserva una copia del primer equipo participante.
 - *Condiciones de la variable:* Sus Pokémon pueden perder salud durante la batalla. Sus cambios de salud no afectan al equipo original.
- **Segundo equipo (secondTeam), equipo Pokémon.** Conserva una copia del segundo equipo participante.

- *Condiciones de la variable:* Sus Pokémon pueden perder salud durante la batalla. Sus cambios de salud no afectan al equipo original.
- **Acontecimientos de la batalla (battleHistory), secuencia cronológica de cadenas de caracteres.** Registra los ataques, los debilitamientos y el resultado final.
 - *Condiciones de la variable:* Permanece vacía mientras la batalla no se haya ejecutado. Conserva los acontecimientos en el orden en que ocurrieron.
- **Nombre del equipo ganador (winnerTeamName), cadena de caracteres.** Identifica al equipo que ganó la batalla.
 - *Condiciones de la variable:* Permanece vacío mientras no se haya determinado un ganador. Permanece vacío cuando la batalla termina sin ganador.
- **Nombre del entrenador ganador (winnerTrainerName), cadena de caracteres.** Identifica al propietario del equipo ganador.
 - *Condiciones de la variable:* Permanece vacío mientras no se haya determinado un ganador. Permanece vacío cuando la batalla termina sin ganador.
- **Estado de finalización (finished), booleano.** Indica si la batalla ya fue ejecutada completamente.
 - *Condiciones de la variable:* Su valor inicial es falso. Su valor es verdadero después de finalizar la batalla. Vuelve a ser falso cuando la batalla es restablecida.

Interfaz

- **crearBattle(firstTrainerName, firstTeam, secondTrainerName, secondTeam): Battle.** Inicia una batalla con los entrenadores y equipos recibidos.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se conservan los nombres de los dos entrenadores. Se conserva una copia de cada equipo. Las copias mantienen la salud que tenían los Pokémon al iniciar la batalla. Los acontecimientos quedan vacíos. Los nombres del equipo y del entrenador ganador quedan vacíos. El estado de finalización queda en falso.
- **getTypeMultiplier(attackerType, defenderType): numérico decimal.** Determina la efectividad del tipo atacante frente al tipo defensor.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna 2.0 cuando la relación entre los tipos es fuerte. Retorna 0.5 cuando la relación entre los tipos es débil. Retorna 1.0 cuando la combinación no tiene una relación especial. Los datos de la batalla conservan sus valores.
- **calculateDamage(attacker, defender): numérico entero.** Calcula cuántos puntos de salud debe perder el Pokémon defensor según el ataque, la defensa y la relación entre sus tipos.
 - *Precondiciones:* Ninguna.

- *Poscondiciones:* Calcula el daño multiplicando el ataque por la efectividad del tipo y restando la mitad de la defensa. La parte decimal del resultado calculado se descarta antes de retornar el daño. Retorna como mínimo un punto de daño. La salud del atacante conserva su valor. La salud del defensor conserva su valor. Los acontecimientos de la batalla conservan sus valores.
- **executeAttack(attacker, defender): vacío.** Aplica al defensor el daño causado por el atacante y registra lo ocurrido.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* La salud del defensor disminuye según el daño calculado. La salud del defensor no queda por debajo de cero. La salud y las estadísticas del atacante conservan sus valores. Se agrega un acontecimiento con los nombres de los Pokémon y el daño causado. Si el defensor queda sin salud, se agrega otro acontecimiento que informa su debilitamiento. Los nuevos acontecimientos quedan después de los registrados anteriormente.
- **getFirstTrainerName(): cadena de caracteres.** Permite conocer el nombre del entrenador propietario del primer equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre del primer entrenador. Los datos de la batalla conservan sus valores.
- **getSecondTrainerName(): cadena de caracteres.** Permite conocer el nombre del entrenador propietario del segundo equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre del segundo entrenador. Los datos de la batalla conservan sus valores.
- **getFirstTeam(): equipo de solo lectura.** Permite consultar el estado de la copia del primer equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna acceso de solo lectura al primer equipo. Los datos de la batalla conservan sus valores.
- **getSecondTeam(): equipo de solo lectura.** Permite consultar el estado de la copia del segundo equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna acceso de solo lectura al segundo equipo. Los datos de la batalla conservan sus valores.
- **isFinished(): booleano.** Determina si la batalla ya fue ejecutada completamente.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna verdadero cuando la batalla ha finalizado. Retorna falso cuando la batalla todavía no se ha ejecutado o fue restablecida. Los datos de la batalla conservan sus valores.

- **getWinnerTeamName(): cadena de caracteres.** Permite conocer el nombre del equipo ganador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre del equipo ganador cuando existe. Retorna una cadena vacía cuando todavía no existe un ganador. Retorna una cadena vacía cuando la batalla terminó sin ganador. Los datos de la batalla conservan sus valores.
- **getWinnerTrainerName(): cadena de caracteres.** Permite conocer el nombre del entrenador propietario del equipo ganador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre del entrenador ganador cuando existe. Retorna una cadena vacía cuando todavía no existe un ganador. Retorna una cadena vacía cuando la batalla terminó sin ganador. Los datos de la batalla conservan sus valores.
- **getBattleHistory(): secuencia de cadenas de caracteres de solo lectura.** Permite consultar todos los acontecimientos registrados durante la batalla.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna los acontecimientos en el orden en que ocurrieron. Retorna una secuencia vacía cuando todavía no se ha registrado ningún acontecimiento. Los acontecimientos no pueden modificarse mediante el acceso retornado.
- **runBattle(): booleano.** Ejecuta el enfrentamiento hasta que uno de los equipos se quede sin Pokémon disponibles. El Pokémon más rápido ataca primero y, si las velocidades son iguales, comienza el Pokémon del primer equipo.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Si la batalla ya había finalizado, retorna falso. Cuando retorna falso, los equipos, los acontecimientos y el ganador conservan sus valores. Si la batalla no había finalizado, la ejecuta completamente y retorna verdadero. Cuando retorna verdadero, el estado de finalización queda en verdadero. Cuando retorna verdadero, los cambios de salud quedan aplicados únicamente sobre las copias de los equipos. Cuando uno de los equipos conserva Pokémon disponibles, sus nombres de equipo y entrenador quedan registrados como ganadores. Cuando los dos equipos comienzan derrotados, los nombres del ganador permanecen vacíos. Los acontecimientos quedan registrados en el orden en que ocurrieron.
- **resetBattle(): vacío.** Restablece la batalla para permitir que vuelva a ejecutarse.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* La salud actual de todos los Pokémon de las dos copias queda igual a su salud máxima. Los acontecimientos anteriores son eliminados. El nombre del equipo ganador queda vacío. El nombre del entrenador ganador queda vacío. El estado de finalización queda en falso. Los nombres de los entrenadores conservan sus valores. Los integrantes y el orden de los equipos conservan sus valores.

- **printBattleHistory(): vacío.** Presenta los equipos participantes y los acontecimientos registrados durante la batalla.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se muestran los nombres de los dos equipos participantes. Si no hay acontecimientos, se informa que la batalla todavía no ha sido ejecutada. Si existen acontecimientos, se muestran en el orden en que ocurrieron. Los datos de la batalla conservan sus valores.

4.5. TAD TeamResult

Representa las estadísticas obtenidas por un equipo durante la comparación más reciente entre dos entrenadores. Conserva la identificación del entrenador y del equipo, junto con sus batallas disputadas, victorias y porcentaje de victorias.

Estado

- **Nombre del entrenador (trainerName), cadena de caracteres.** Identifica al entrenador propietario del equipo evaluado.
- **Nombre del equipo (teamName), cadena de caracteres.** Identifica al equipo evaluado.
- **Batallas disputadas (battles), numérico entero.** Indica cuántas veces participó el equipo durante la comparación.
 - *Condiciones de la variable:* Su valor inicial es cero. Su valor no puede ser negativo.
- **Victorias (victories), numérico entero.** Indica cuántas batallas ganó el equipo.
 - *Condiciones de la variable:* Su valor inicial es cero. Su valor no puede ser negativo. Su valor no puede superar la cantidad de batallas disputadas.
- **Porcentaje de victorias (winPercentage), numérico decimal.** Representa el porcentaje de batallas ganadas por el equipo.
 - *Condiciones de la variable:* Su valor inicial es cero. Su valor se encuentra entre cero y cien.

Interfaz

- **crearTeamResult(trainerName, teamName): TeamResult.** Inicializa el seguimiento de un equipo dentro de una comparación y lo asocia con su entrenador.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Se obtiene un **TeamResult** con el nombre del entrenador recibido. Se obtiene un **TeamResult** con el nombre del equipo recibido. Las batallas disputadas quedan en cero. Las victorias quedan en cero. El porcentaje de victorias queda en cero.
- **getTrainerName(): cadena de caracteres.** Permite conocer el nombre del entrenador registrado en el resultado.
 - *Precondiciones:* Ninguna.

- *Poscondiciones:* Retorna el nombre del entrenador almacenado. Los datos del resultado conservan sus valores.
- **setTrainerName(trainerName): vacío.** Modifica el nombre del entrenador registrado en el resultado.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* El nombre del entrenador queda reemplazado por el valor recibido. Los demás datos del resultado conservan sus valores.
- **getTeamName(): cadena de caracteres.** Permite conocer el nombre del equipo registrado en el resultado.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna el nombre del equipo almacenado. Los datos del resultado conservan sus valores.
- **setTeamName(teamName): vacío.** Modifica el nombre del equipo registrado en el resultado.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* El nombre del equipo queda reemplazado por el valor recibido. Los demás datos del resultado conservan sus valores.
- **getBattles(): numérico entero.** Permite conocer cuántas batallas disputó el equipo durante la comparación.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la cantidad de batallas almacenada. Los datos del resultado conservan sus valores.
- **setBattles(battles): vacío.** Modifica la cantidad de batallas disputadas por el equipo.
 - *Precondiciones:* **battles** es un valor no negativo. **battles** no es menor que la cantidad de victorias almacenada.
 - *Poscondiciones:* La cantidad de batallas queda reemplazada por el valor recibido. Los demás datos del resultado conservan sus valores.
- **getVictories(): numérico entero.** Permite conocer cuántas batallas ganó el equipo durante la comparación.
 - *Precondiciones:* Ninguna.
 - *Poscondiciones:* Retorna la cantidad de victorias almacenada. Los datos del resultado conservan sus valores.
- **setVictories(victories): vacío.** Modifica la cantidad de victorias obtenidas por el equipo.
 - *Precondiciones:* **victories** es un valor no negativo. **victories** no supera la cantidad de batallas almacenada.
 - *Poscondiciones:* La cantidad de victorias queda reemplazada por el valor recibido. Los demás datos del resultado conservan sus valores.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Declaración <code>enumclassPokemonType</code> y valores como <code>PokemonType::FIRE</code> y <code>PokemonType::WATER</code> .	<code>Pokemon.h</code> , <code>Pokemon.cxx</code> , <code>Battle.h</code> , <code>Battle.cxx</code> y <code>main.cxx</code> .	No se conocía qué era un <code>enum</code> , por qué <code>PokemonType</code> aparecía como un tipo de dato ni por qué sus valores se escribían usando <code>PokemonType::</code> .	Consulta realizada al profesor y documentación de Microsoft Learn sobre enumeraciones en C++. [4]	Se comprendió que <code>enumclassPokemonType</code> crea un tipo de dato cuyos valores posibles están definidos en el mismo código: <code>FIRE</code> , <code>WATER</code> , <code>GRASS</code> , <code>ELECTRIC</code> , <code>GROUND</code> y <code>FLYING</code> . Por esta razón, el atributo <code>type</code> de <code>Pokemon</code> no recibe cualquier texto o número, sino uno de esos valores. La parte <code>PokemonType::</code> indica que el valor pertenece específicamente a esa enumeración; por ejemplo, <code>PokemonType::FIRE</code> representa el valor <code>FIRE</code> del tipo <code>PokemonType</code> . En <code>main.cxx</code> , <code>stringToPokemonType()</code> convierte el texto leído de <code>trainers.txt</code> , como <code>FIRE</code> , en el valor correspondiente de la enumeración. Después, <code>Battle.cxx</code> compara esos valores para determinar el multiplicador de daño. Así, el archivo de entrada utiliza texto, pero los objetos del simulador almacenan y comparan valores de <code>PokemonType</code> .

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Lista de inicialización de los constructores: <code>name(pokemonName), type(pokemonType), firstTeam(teamOne)</code> y <code>finished(false)</code> .	<code>Pokemon.cxx</code> , <code>Team.cxx</code> , <code>Trainer.cxx</code> y <code>Battle.cxx</code> .	No se comprendía por qué, después de los parámetros del constructor, aparecía : seguido de atributos con valores entre paréntesis. Expresiones como <code>name(pokemonName)</code> parecían llamadas a funciones.	Microsoft Learn, documentación sobre constructores y listas de inicialización en C++. [5]	Se comprendió que lo escrito después de : no corresponde a llamadas independientes a funciones, sino a la lista que inicializa los atributos antes de ejecutar el cuerpo del constructor. A la izquierda aparece el atributo y entre paréntesis se indica el valor inicial que recibirá. En <code>Pokemon.cxx</code> , <code>name(pokemonName)</code> guarda el parámetro <code>pokemonName</code> en el atributo <code>name</code> , mientras que <code>currentHealth(pokemonHealth)</code> hace que el Pokémon comience con la salud actual igual a la máxima. En <code>Battle.cxx</code> , <code>firstTeam(teamOne)</code> y <code>secondTeam(teamTwo)</code> crean las copias de los equipos que serán usadas durante la batalla. Por eso los cambios de salud del enfrentamiento se aplican a esas copias y no directamente a los equipos guardados por los entrenadores. Finalmente, <code>winnerTeamName("")</code> , <code>winnerTrainerName("")</code> y <code>finished(false)</code> indican que una batalla recién creada todavía no tiene ganador ni ha finalizado.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Declaraciones <code>std::list<Pokemon>::const_iterator</code> y <code>getNotFoundPosition()</code> ; y <code>std::list<Pokemon>::const_iterator</code> y <code>findPokemon(const std::string& pokemonName) const</code> ;	<code>Team.h</code> y <code>Team.cxx</code> .	No se comprendía si toda la expresión correspondía a una función, qué parte era el tipo retornado ni por qué aparecía nuevamente <code>const</code> al final. Tampoco estaba clara la razón para tener dos versiones de <code>getNotFoundPosition()</code> y <code>findPokemon()</code> .	Diapositivas 02.2-Secuencias(3).pptx y 03-Vectores y Listas(3).pptx; documentación de Microsoft Learn sobre funciones miembro constantes. [10, 6]	Se comprendió que toda la expresión sí declara una función. En <code>findPokemon</code> , <code>std::list<Pokemon>::const_iterator</code> es el tipo que retorna, <code>findPokemon</code> es el nombre, <code>pokemonName</code> es el parámetro y el <code>const</code> final indica que la función no modifica el equipo sobre el que se invoca. El valor retornado es un iterador constante, por lo que permite consultar el Pokémon encontrado, pero no modificarlo a través de ese iterador. La clase también tiene otra versión que retorna <code>iterator</code> y no posee el <code>const</code> final; esa versión puede trabajar con un equipo modificable y retornar una posición que permita alterar al Pokémon. El compilador selecciona la versión adecuada según si el objeto <code>Team</code> es constante o modificable. Actualmente, los cuerpos de <code>findPokemon()</code> corresponden a los TODO #01 y TODO #02, por lo que todavía retornan directamente <code>getNotFoundPosition()</code> y aún no realizan el recorrido solicitado.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Uso de iteradores mediante <code>begin()</code> , <code>end()</code> , <code>-></code> y <code>++</code> , por ejemplo en <code>teamPosition->getName()</code> y <code>++teamPosition</code> .	<code>Team.cxx</code> , <code>Trainer.cxx</code> , <code>Battle.cxx</code> , <code>BattleSimulator.cxx</code> y <code>main.cxx</code> .	Aunque el concepto de iterador se había visto en clase, no estaba claro cómo leer estas instrucciones dentro de un recorrido real ni qué hacían la flecha y el incremento sobre el iterador.	Diapositivas 02.2-Secuencias(3).pptx y 03-Vectores y Listas(3).pptx; documentación de Microsoft Learn sobre iteradores. [10, 7]	Se comprendió que el iterador representa una posición dentro de una estructura lineal. <code>begin()</code> entrega la posición del primer elemento y <code>end()</code> representa la posición que está después del último; por eso no se procesa <code>end()</code> , sino que se utiliza para saber cuándo terminar el recorrido. El operador <code>++</code> hace avanzar el iterador al siguiente elemento. La flecha <code>-></code> permite acceder a una función o atributo del objeto ubicado en la posición actual y equivale a acceder primero al objeto señalado por el iterador. En <code>Trainer::findTeam()</code> , <code>teamPosition->getName()</code> consulta el nombre del equipo actual y <code>++teamPosition</code> avanza al siguiente. En <code>BattleSimulator::printComparisonResults()</code> , expresiones como <code>resultPosition->trainerName</code> acceden a los datos del <code>TeamResult</code> actual. El mismo esquema se utiliza para recorrer Pokémon, equipos, resultados, eventos y batallas sin perder el orden en el que están almacenados.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Valor -1 retornado por <code>findTeam()</code> , <code>findTrainer()</code> y <code>findComparisonResult()</code> .	<code>Trainer.h</code> , <code>Trainer.cxx</code> , <code>BattleSimulator.h</code> y <code>BattleSimulator.cxx</code> .	No se comprendía qué ocurría después de retornar -1: si ese número se mostraba en la terminal, si se intentaba utilizar como posición o cómo sabía otra función que la búsqueda había fallado.	Código suministrado del taller, comentarios de las funciones y diapositiva 02.2-Secuencias(3).pptx. [10]	Se comprendió que -1 es una señal de búsqueda fallida y no una posición válida del vector, debido a que sus posiciones comienzan en cero. Retornarlo no hace que el número aparezca automáticamente en la terminal. El valor vuelve a la función que realizó la llamada y allí se compara antes de acceder al vector. Por ejemplo, <code>containsTeam()</code> evalúa <code>findTeam(teamName) != -1</code> : si la búsqueda devuelve -1, el resultado es <code>false</code> . En <code>removeTeam()</code> , el valor se guarda en <code>position</code> y, si es -1, la función retorna <code>false</code> antes de intentar eliminar un elemento. <code>BattleSimulator</code> sigue el mismo procedimiento con los entrenadores y los resultados de comparación. Por eso el tipo retornado es <code>int</code> , ya que debe representar tanto posiciones desde cero como el valor negativo utilizado para indicar que no se encontró el elemento.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Elección entre <code>iterator</code> y <code>const_iterator</code> , especialmente en <code>findTeam()</code> y <code>restoreAllTeams()</code> .	<code>Team.h</code> , <code>Team.cxx</code> , <code>Trainer.cxx</code> , <code>Battle.cxx</code> , <code>BattleSimulator.cxx</code> y <code>main.cxx</code> .	No se comprendía por qué <code>findTeam()</code> recorría los equipos con un <code>const_iterator</code> , mientras <code>restoreAllTeams()</code> necesitaba un <code>iterator</code> , si las dos funciones avanzaban por el mismo vector.	Diapositivas 02.2-Secuencias(3).pptx y 03-Vectores y Listas(3).pptx; documentación de Microsoft Learn sobre iteradores y uso de <code>const</code> . [10, 6, 7]	Se comprendió que la diferencia no depende de que el iterador avance con <code>++</code> , porque tanto un <code>iterator</code> como un <code>const_iterator</code> pueden recorrer la estructura. La elección depende de lo que se hará con el elemento encontrado. <code>findTeam()</code> solamente consulta <code>teamPosition->getName()</code> para comparar nombres; por eso utiliza un <code>const_iterator</code> , que impide modificar el equipo durante la búsqueda. En cambio, <code>restoreAllTeams()</code> llama <code>teamPosition->restoreTeam()</code> , operación que cambia la salud de los Pokémon del equipo. Por eso necesita un <code>iterator</code> modificable. Esta decisión se repite en otros archivos: las funciones de impresión recorren los elementos con iteradores constantes, mientras las operaciones que eliminan, reorganizan o cambian el estado necesitan iteradores modificables. Así, el tipo de iterador funciona como una restricción que evita cambios accidentales durante las operaciones de consulta.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Uso de <code>push_back</code> , por ejemplo: <code>teamList.push_back(team)</code> , <code>trainerList.push_back(trainer)</code> y <code>comparisonResults.push_back(result)</code> .	<code>Trainer.cxx</code> y <code>BattleSimulator.cxx</code> .	No se comprendía si <code>push_back</code> guardaba el objeto completo, solamente una referencia o su dirección de memoria. Tampoco estaba claro qué pasaba con el elemento almacenado cuando terminaba la función.	Diapositiva 03-Vectores y Listas(3).pptx y documentación de Microsoft Learn sobre <code>vector::push_back</code> . [10, 8]	Se comprendió que <code>push_back</code> agrega un elemento al final del vector y que, en estas instrucciones, se guarda una copia del objeto recibido. En <code>Trainer::addTeam()</code> , primero se comprueba que no exista un equipo con el mismo nombre y luego <code>teamList.push_back(team)</code> copia el equipo dentro del vector del entrenador. En <code>BattleSimulator::addTrainer()</code> , <code>trainerList.push_back(trainer)</code> realiza el mismo proceso con el entrenador. Por eso el simulador conserva el objeto aunque termine la función donde fue agregado. En la comparación, <code>comparisonResults.push_back(result)</code> almacena una copia de la estructura <code>TeamResult</code> , incluidos el entrenador, el equipo y los contadores iniciales. Además, <code>bestTeams.push_back(resultPosition->teamName)</code> copia el nombre seleccionado dentro del vector que será retornado. Los parámetros se reciben mediante referencia constante para evitar una copia innecesaria al entrar en la función y para impedir que se modifique el argumento original; la copia que debe quedar almacenada se realiza al llamar <code>push_back</code> .

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Manipuladores de salida <code>std::left</code> , <code>std::setw</code> , <code>std::fixed</code> y <code>std::setprecision</code> .	<code>Pokemon.cxx</code> y <code>BattleSimulator.cxx</code> .	No se conocía para qué servían estas instrucciones, en qué casos se utilizaban ni por qué aparecían al imprimir las tarjetas de los Pokémon y los resultados de las comparaciones. Tampoco estaba claro si cambiaban los valores almacenados.	Microsoft Learn, documentación sobre las funciones de <code><iomanip></code> y el control del formato de salida. [9]	Se comprendió que estas instrucciones se utilizan para que la información no cambie de posición cada vez que se imprime un dato con una longitud diferente. En <code>Pokemon::print()</code> , <code>std::setw(20)</code> reserva veinte espacios para el siguiente dato y <code>std::left</code> hace que este comience desde la izquierda. Por ejemplo, después de imprimir Pikachu, se dejan los espacios faltantes antes del borde . Así, el borde de la tarjeta permanece alineado aunque cambien el nombre, el tipo o las estadísticas. <code>setw</code> aparece varias veces porque solamente se aplica al dato que se escribe inmediatamente después. En <code>BattleSimulator::printComparisonResults()</code> ocurre algo parecido: se reserva un ancho diferente para cada columna, de modo que las batallas y victorias aparezcan debajo de su encabezado aunque nombres como Ash y ElectricWave tengan longitudes distintas. Finalmente, <code>std::fixed</code> indica que el porcentaje se mostrará como un número decimal normal y <code>std::setprecision(2)</code> deja dos cifras después del punto. Por eso 66.666... aparece como 66.67 y 100 como 100.00. Estas instrucciones no cambian los valores guardados; solamente organizan su presentación.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
Inicialización y modificación de los atributos privados de Battle sin funciones setter. Ejemplo: <code>firstTeam(teamOne)</code> , <code>winnerTeamName("")</code> y <code>finished(false)</code> .	<code>Battle.h</code> , <code>Battle.cxx</code> y <code>BattleSimulator.cxx</code> .	No se comprendía cómo debían asignarse o cambiarse los datos de una batalla porque la clase no tiene setters para los equipos, los entrenadores, el ganador ni el atributo <code>finished</code> .	Código suministrado del taller y diapositiva 02.1-TAD(3).pptx sobre estado e interfaz de un TAD. [11]	Se comprendió que los setters no son necesarios para crear la batalla porque su estado inicial se establece directamente en el constructor. Los nombres de los entrenadores y las copias de los equipos se reciben como parámetros, mientras <code>winnerTeamName</code> y <code>winnerTrainerName</code> comienzan como cadenas vacías y <code>finished</code> comienza en <code>false</code> . Después de construir el objeto, estos atributos no pueden cambiarse libremente desde otras clases porque son privados y la interfaz no ofrece setters. Esto evita, por ejemplo, que <code>BattleSimulator</code> asigne un ganador sin ejecutar el enfrentamiento. Los cambios válidos deben realizarse desde las operaciones de la misma clase <code>Battle</code> , que sí tienen acceso a sus atributos privados. De acuerdo con la interfaz y los comentarios del código, <code>runBattle()</code> debe modificar internamente la salud de las copias de los equipos, registrar los acontecimientos, establecer el ganador y cambiar <code>finished</code> a <code>true</code> . Por su parte, <code>resetBattle()</code> restaura los equipos, limpia el historial y los nombres del ganador, y vuelve a colocar <code>finished</code> en <code>false</code> . Actualmente <code>runBattle()</code> y <code>executeAttack()</code> todavía corresponden a los TODO #07 y TODO #06, por lo que esos cambios deben implementarse más adelante. Por tanto, la ausencia de setters no es un error: el estado se controla mediante las operaciones propias del TAD.

Elemento consultado	Archivo donde aparece	Duda inicial	Fuente consultada	Explicación obtenida
---------------------	-----------------------	--------------	-------------------	----------------------

7. Caso de prueba básico

¿Cuántos entrenadores hay registrados?

2 entrenadores registrados Ash y Misty

¿Cuántos equipos tiene cada entrenador?

Ash cuenta con 2 equipos, Misty cuenta con 3 equipos

¿Qué operaciones serán ejecutadas y en qué orden?

Primero se ejecuta un PRINT, el cual muestra por pantalla la información de todos los entrenadores que fueron cargados desde trainers.txt, luego se realiza un combate entre Ash con el equipo de Johto contra Misty con el equipo de Cerulean, luego se ejecuta el comando COMPARE el cual enfrenta cada equipo de Ash contra cada equipo de Misty, ejecuta 6 batallas en un orden específico, registra sus resultados y posteriormente calcula las estadísticas de victorias de cada equipo. Por ultimo realiza una ultima batalla entre Ash con el equipo de Johto y Ash con el equipo de Kanto.

¿Qué equipos participarán en la batalla individual?

En las batallas individuales, los comandos BATTLE, participan los equipos Johto, Cerulean y Kanto en caso de que Ash se puede enfrentar a si mismo

¿Qué Pokémon debería atacar primero en cada enfrentamiento? Justifique usando su velocidad.

- **Combate 1:** Ataca de primeras Staryu con velocidad de 85
- **Combate 2:** Ataca de primeras Pikachu con velocidad de 90
- **Combate 3:** Ataca de primeras Pikachu con velocidad de 90
- **Combate 4:** Ataca de primeras Pikachu con velocidad de 90
- **Combate 5:** Ataca de primeras Staryu con velocidad de 85
- **Combate 6:** Ataca de primeras Cyndaquil con velocidad de 65
- **Combate 7:** Ataca de primeras Cyndaquil con velocidad de 65
- **Combate 8:** Ataca de primeras Pikachu con velocidad de 90

7. ¿Cómo influyen los tipos de los Pokémon en el daño?

El tipo modifica directamente el multiplicador utilizado para calcular el daño. La función `getTypeMultiplier()` contempla tres situaciones principales:

- 2.0: el ataque es muy efectivo.
- 0.5: el ataque es poco efectivo.
- 1.0: no existe una modificación de efectividad explícita. El daño se calcula mediante:

$$\text{Daño} = \text{Ataque} * \text{Multiplicador} - (\text{Defensa}/2)$$

Posteriormente, si el resultado es menor que 1, el programa establece el daño mínimo en 1 punto.

Por ejemplo, Cyndaquil es de tipo Fuego y Staryu es de tipo Agua. Para un ataque de Fuego contra Agua, el multiplicador es 0.5. Por ello, el ataque de Cyndaquil resulta poco efectivo:

$$52(0,5) - (55/2)$$

$$26 - 27,5 = -1,5$$

Como el programa garantiza un daño mínimo de 1, Cyndaquil solamente causaría 1 punto de daño a Staryu.

En sentido contrario, Saryu es de tipo Agua y Cyndaquil es de tipo Fuego. Agua es un tipo efectivo contra Fuego, por lo que:

$$45(2) - (43/2)$$

$$90 - 21,5 = 68,5$$

Al convertir el resultado a entero, el daño sería de 68 puntos.

Este ejemplo demuestra que el tipo tiene una influencia considerable sobre el resultado de los enfrentamientos.

8. ¿Qué entrenador y equipo se espera que ganen la batalla individual?

Para la primera batalla, Misty y su equipo Cerulean presentan una ventaja considerable.

La principal razón es la combinación entre tipo y velocidad. Saryu tiene una velocidad de 85, superior a los 65 de Cyndaquil y a los 43 de Totodile. Además, sus ataques de tipo Agua son especialmente efectivos contra Cyndaquil, que es de tipo Fuego.

El primer enfrentamiento, por tanto, favorece claramente a Saryu:

- Saryu ataca primero.
- Agua es fuerte contra Fuego.
- Cyndaquil tiene una defensa de 43.
- Cyndaquil solamente causa daño reducido contra Saryu debido a la desventaja Fuego $\rightarrow$ Agua.

Una vez derrotado Cyndaquil, Totodile representa una amenaza mayor debido a su mayor ataque y defensa. Sin embargo, Saryu conserva la ventaja de velocidad.

Por estas razones, la expectativa inicial es que Misty gane la batalla con el equipo Cerulean. De hecho, siguiendo literalmente las estadísticas y fórmulas implementadas, Cerulean termina imponiéndose en esta batalla.

9. ¿Cuántas batallas debería generar la comparación de entrenadores?

La operación: `COMPAREAshMisty` debe enfrentar cada equipo de Ash contra cada equipo de Misty. Ash posee:

2 equipos

Misty posee:

3 equipos

Por lo tanto:

$$2 * 3 = 6$$

La comparación debe generar 6 batallas:

El orden se debe a los recorridos anidados implementados en `compareTrainers()`: primero se mantiene Kanto mientras se recorren todos los equipos de Misty y posteriormente se hace lo mismo con Johto.

10. ¿Cuántas batallas deberían aparecer en el historial al finalizar?

Antes de ejecutar las operaciones, el historial está vacío. La primera operación, `PRINT`, no agrega ninguna batalla. Posteriormente:

- `BATTLEAshJohtoMistyCerulean` $\rightarrow$ 1 batalla.

#	Equipo de Ash	Equipo de Misty
1	Kanto	Cerulean
2	Kanto	ElectricWave
3	Kanto	GreenGarden
4	Johto	Cerulean
5	Johto	ElectricWave
6	Johto	GreenGarden

- COMPAREAshMisty $\rightarrow$ 6 batallas.
- BATTLEAshJohtoAshKanto $\rightarrow$ 1 batalla. Por lo tanto:

$$1 + 6 + 1 = 8$$

Al finalizar todas las operaciones deberían existir 8 batallas en `battleHistory`. El orden sería:

Posición	Batalla
1	Ash Johto vs. Misty Cerulean
2	Ash Kanto vs. Misty Cerulean
3	Ash Kanto vs. Misty ElectricWave
4	Ash Kanto vs. Misty GreenGarden
5	Ash Johto vs. Misty Cerulean
6	Ash Johto vs. Misty ElectricWave
7	Ash Johto vs. Misty GreenGarden
8	Ash Johto vs. Ash Kanto

Es importante señalar que la batalla entre Ash Johto y Misty Cerulean aparece dos veces: la primera corresponde a la instrucción `BATTLE` independiente y la segunda es una de las seis combinaciones generadas por `COMPARE`.

11. ¿Cuáles equipos se espera que obtengan los mejores resultados?

Al analizar las estadísticas de los Pokémon y las relaciones de tipo implementadas, los equipos que presentan mejores perspectivas son principalmente los equipos de Misty.

En particular:

Cerulean

Presenta una combinación favorable porque Staryu posee una velocidad de 85 y es de tipo Agua. Esto le proporciona una ventaja importante contra Pokémon de tipo Fuego y le permite atacar antes que varios de sus posibles oponentes.

GreenGarden

También presenta una combinación interesante debido a Diglett, que posee 95 de velocidad, siendo uno de los Pokémon más rápidos del conjunto. Además, el tipo Tierra resulta especialmente favorable contra Pokémon Eléctricos y de Fuego.

ElectricWave

Cuenta con Zapdos, que tiene 100 de velocidad y 90 de ataque, convirtiéndolo en uno de los Pokémon con mejores estadísticas ofensivas. Además, el tipo Eléctrico es muy efectivo contra Pokémon de tipo Agua y el tipo Volador resulta vulnerable frente a ataques Eléctricos.

Por su parte, Johto también presenta potencial, especialmente por Totodile y su ataque de 65, mientras que Kanto posee una gran cantidad de Pokémon y varias estadísticas elevadas. Sin embargo, el desempeño depende de las combinaciones específicas de tipos y de la velocidad de los Pokémon.

Considerando las seis batallas de la comparación y aplicando las fórmulas implementadas en el programa, la expectativa general favorece a los equipos de Misty. En particular, Cerulean y GreenGarden presentan los mejores resultados esperados, mientras que ElectricWave también posee un potencial elevado gracias a Zapdos.

7.1. Análisis del caso de prueba básico del simulador de batallas Pokémon

¿Cuántos entrenadores se encuentran registrados?

En el archivo de configuración del simulador se encuentran registrados dos entrenadores: Ash y Misty.

¿Cuántos equipos tiene cada entrenador? Por consiguiente, el simulador

Entrenador	Número de equipos
Ash	2
Misty	3

cuenta con cinco equipos registrados en total.

¿Qué operaciones serán ejecutadas y en qué orden?

Las operaciones se ejecutan de manera secuencial, siguiendo el orden establecido en el archivo de operaciones. En primer lugar, se ejecuta la operación **PRINT**, cuya función es mostrar la información correspondiente a los entrenadores y sus equipos. Posteriormente, se ejecuta una batalla entre Ash, con su equipo Johto, y Misty, con su equipo Cerulean. A continuación, se ejecuta la operación **COMPAREAshMisty**. Esta operación enfrenta cada uno de los equipos de Ash contra cada uno de los equipos de Misty, generando seis enfrentamientos. Finalmente, se ejecuta una última batalla entre Ash, con el equipo Johto, y Ash, con el equipo Kanto.

Orden	Operación	Enfrentamiento o acción
1	PRINT	Muestra la información de los entrenadores y equipos
2	BATTLE	Ash — Johto vs. Misty — Cerulean
3	COMPARE	Ash — Kanto vs. Misty — Cerulean
4	COMPARE	Ash — Kanto vs. Misty — ElectricWave
5	COMPARE	Ash — Kanto vs. Misty — GreenGarden
6	COMPARE	Ash — Johto vs. Misty — Cerulean
7	COMPARE	Ash — Johto vs. Misty — ElectricWave
8	COMPARE	Ash — Johto vs. Misty — GreenGarden
9	BATTLE	Ash — Johto vs. Ash — Kanto

La operación **COMPARE** genera seis enfrentamientos debido a que Ash posee dos equipos y Misty posee tres: $2 \times 3 = 6$.

¿Qué equipos participarán en la batalla individual?

La primera batalla individual corresponde al enfrentamiento entre Ash, con su equipo Johto, y Misty, con su equipo Cerulean. Por lo tanto, los equipos participantes son: Ash — Johto y Misty — Cerulean. Asimismo, la última operación BATTLE enfrenta los equipos Johto y Kanto de Ash.

¿Qué Pokémon debería atacar primero en cada enfrentamiento? Justifique utilizando su velocidad. El orden de ataque se determina a partir de la velocidad de los Pokémon participantes. El Pokémon con mayor velocidad debe realizar el primer ataque.

Enfrentamiento	Primer atacante	Velocidad
1	Saryu	85
2	Pikachu	90
3	Pikachu	90
4	Pikachu	90
5	Saryu	85
6	Cyndaquil	65
7	Cyndaquil	65
8	Pikachu	90

Estos resultados se explican por la comparación de las estadísticas de velocidad de los Pokémon que participan en cada enfrentamiento. En consecuencia, el Pokémon con el valor de velocidad más elevado obtiene la prioridad para realizar el ataque.

7. ¿Cómo influyen los tipos de los Pokémon en el daño? El tipo modifica directamente el multiplicador utilizado para calcular el daño. La función `getTypeMultiplier()` contempla tres situaciones principales:

- 2.0: el ataque es muy efectivo.
- 0.5: el ataque es poco efectivo.
- 1.0: no existe una modificación de efectividad explícita.

El daño se calcula mediante la siguiente expresión:

$$\text{Daño} = \text{Ataque} \times \text{Multiplicador} - \frac{\text{Defensa}}{2}$$

Posteriormente, si el resultado es menor que 1, el programa establece el daño mínimo en 1 punto.

Por ejemplo, Cyndaquil es de tipo Fuego y Saryu es de tipo Agua. Para un ataque de Fuego contra Agua, el multiplicador es 0.5. Por ello, el ataque de Cyndaquil resulta poco efectivo:

$$52(0,5) - \frac{55}{2} = 26 - 27,5 = -1,5$$

Como el programa garantiza un daño mínimo de 1, Cyndaquil solamente causaría 1 punto de daño a Saryu.

En sentido contrario, Saryu es de tipo Agua y Cyndaquil es de tipo Fuego. Agua es un tipo efectivo contra Fuego, por lo que:

$$45(2) - \frac{43}{2} = 90 - 21,5 = 68,5$$

Al convertir el resultado a entero, el daño sería de 68 puntos.

Este ejemplo demuestra que el tipo tiene una influencia considerable sobre el resultado de los enfrentamientos.

¿Qué entrenador y equipo se espera que ganen la batalla individual?

Para la primera batalla, correspondiente al enfrentamiento entre Ash — Johto y Misty — Cerulean, se espera que Misty, con su equipo Cerulean, obtenga la victoria.

Esta expectativa se fundamenta principalmente en las diferencias de velocidad y en la efectividad de los tipos. Saryu posee una velocidad de 85, superior a la de Cyndaquil, que es de 65, y a la de Totodile, que es de 43. Además, sus ataques de tipo Agua presentan una ventaja frente a Cyndaquil, que pertenece al tipo Fuego.

En consecuencia, Saryu tiene la posibilidad de atacar primero y producir un daño considerablemente mayor que el que Cyndaquil puede infligirle. Aunque Totodile presenta mejores valores de ataque y defensa que Cyndaquil, Saryu conserva una ventaja de velocidad.

Por estas razones, se espera que Misty y el equipo Cerulean sean los vencedores de esta batalla. De acuerdo con las estadísticas y las fórmulas implementadas, este equipo efectivamente se impone en el enfrentamiento.

¿Cuántas batallas debería generar la comparación de entrenadores?

La operación `COMPAREAshMisty` enfrenta cada equipo de Ash contra cada equipo de Misty. Ash posee 2 equipos, mientras que Misty posee 3 equipos. Por lo tanto, el número de combinaciones posibles es: $2 \times 3 = 6$. La comparación debe generar seis batallas, en el siguiente orden:

N.º	Equipo de Ash	Equipo de Misty
1	Kanto	Cerulean
2	Kanto	ElectricWave
3	Kanto	GreenGarden
4	Johto	Cerulean
5	Johto	ElectricWave
6	Johto	GreenGarden

Este orden corresponde al recorrido de los equipos mediante los ciclos implementados en la función `compareTrainers()`: primero se comparan todos los equipos de Misty con Kanto y posteriormente se repite el proceso con Johto.

¿Cuántas batallas deberían aparecer en el historial al finalizar?

Al iniciar la ejecución, el historial de batallas se encuentra vacío. La operación `PRINT` no genera ninguna batalla. Posteriormente, la primera operación `BATTLE` genera una batalla, `COMPARE` genera seis y la última operación `BATTLE` genera una batalla adicional.

$$1 + 6 + 1 = 8$$

Por lo tanto, al finalizar la ejecución deberían registrarse ocho batallas en el historial `battleHistory`.

Posición	Enfrentamiento
1	Ash — Johto vs. Misty — Cerulean
2	Ash — Kanto vs. Misty — Cerulean
3	Ash — Kanto vs. Misty — ElectricWave
4	Ash — Kanto vs. Misty — GreenGarden
5	Ash — Johto vs. Misty — Cerulean
6	Ash — Johto vs. Misty — ElectricWave
7	Ash — Johto vs. Misty — GreenGarden
8	Ash — Johto vs. Ash — Kanto

Cabe destacar que el enfrentamiento entre Ash — Johto y Misty — Cerulean aparece dos veces en el historial. La primera corresponde a la instrucción `BATTLE` y la segunda constituye una de las seis combinaciones generadas por `COMPARE`.

¿Cuáles equipos se espera que obtengan los mejores resultados?

De acuerdo con las estadísticas de los Pokémon y las relaciones de tipos implementadas en el simulador, los equipos de Misty presentan, en términos generales, condiciones favorables para obtener buenos resultados.

El equipo Cerulean cuenta con Staryu, cuya velocidad de 85 le permite obtener prioridad frente a varios de sus posibles oponentes. Además, su tipo Agua le proporciona una ventaja frente a Pokémon de tipo Fuego.

El equipo GreenGarden también presenta características favorables, principalmente por Diglett, que posee una velocidad de 95. Su tipo Tierra resulta especialmente relevante frente a Pokémon de tipo Eléctrico y Fuego.

Por su parte, ElectricWave cuenta con Zapdos, que presenta una velocidad de 100 y un ataque de 90, por lo que constituye uno de los Pokémon con mayor capacidad ofensiva dentro de los equipos analizados.

En consecuencia, los equipos que presentan mejores perspectivas son Cerulean y GreenGarden, mientras que ElectricWave también presenta un desempeño potencialmente favorable debido principalmente a las estadísticas de Zapdos. No obstante, el resultado definitivo de cada enfrentamiento depende de la combinación específica de estadísticas, velocidades y tipos de los Pokémon participantes.

7.2. Tabla de pruebas — Caso de prueba básico (punto 14)

Elemento analizado	Resultado esperado	Resultado obtenido	¿Coincide?	Explicación
Batalla individual	Se ejecutan dos batallas puntuales: Johto (Ash) vs. Cerulean (Misty), y Johto (Ash) vs. Kanto (Ash), atacando primero el Pokémon con mayor velocidad en cada turno.	Batalla 1: Saryu ataca primero a Cyndaquil (68 dmg por tipo agua>fuego), termina con Ash-Johto ganador tras debilitar a Saryu y Psyduck. Batalla final: Pikachu ataca primero a Cyndaquil, termina con Ash-Kanto ganador tras debilitar a Cyndaquil, Pikachu, Charizard y, finalmente, a Totodile.	Sí	Ambas instrucciones BATTLE del archivo operations.txt se ejecutaron en el orden esperado, respetando la regla de velocidad para decidir quién ataca primero.
Ganador	Se esperaba Ash-Johto en la primera batalla (Totodile compensa la debilidad de Cyndaquil frente al equipo de Misty) y Ash-Kanto en la batalla final (mejores estadísticas generales frente a Johto).	Ganador batalla 1: Ash - Johto. Ganador batalla final: Ash - Kanto.	Sí	El resultado coincide con el análisis de velocidad y de multiplicador de tipo: Totodile (agua) resultó decisivo contra Cerulean, y Kanto tiene mejores Pokémon de respaldo (Charizard, Squirtle) frente a Johto.
Cantidad de batallas	1 (individual inicial) + 6 (comparación 2×3) + 1 (individual final) = 8 batallas.	El resumen final del programa indica: "Se ejecutaron 8 batallas."	Sí	Coincide exactamente con el conteo esperado; el historial (deque<Battle>) no se reinicia entre operaciones.

Elemento analizado	Resultado esperado	Resultado obtenido	¿Coincide?	Explicación
Resultados de comparación	6 enfrentamientos (Kanto vs. Cerulean, Kanto vs. ElectricWave, Kanto vs. GreenGarden, Johto vs. Cerulean, Johto vs. ElectricWave, Johto vs. GreenGarden). Se esperaba que Kanto y Johto ganaran la mayoría por mejores estadísticas, salvo un posible resultado cerrado contra ElectricWave (Zapdos es fuerte).	Kanto: 3 batallas, 3 victorias (100 %). Johto: 3 batallas, 2 victorias (66.67 %). Cerulean: 2 batallas, 0 victorias (0 %). ElectricWave: 2 batallas, 1 victoria (50 %). GreenGarden: 2 batallas, 0 victorias (0 %).	Sí	Kanto ganó todo, como se esperaba. Johto solo perdió contra ElectricWave, confirmando la sospecha: Zapdos (150 HP, 90 atq, 100 vel) supera a los Pokémon de Johto.
Mejores equipos	Ash: Kanto (único con 100 % de victorias). Misty: ElectricWave (único equipo de Misty con al menos una victoria).	"Mejores equipos de Ash: Kanto/ "Mejores equipos de Misty: ElectricWave"	Sí	getBestTeams() selecciona correctamente el/los equipo(s) con el porcentaje de victorias más alto para cada entrenador.

Referencias

- [1] Nintendo. *¿Qué es Pokémon? Descubre el fenómeno Pokémon*. 2016. Sitio web de Nintendo. Consultado el 24 de agosto de 2026.
- [2] Pokémon Wiki. *Equipo Pokémon*. Página web. Consultada el 24 de agosto de 2026.
- [3] Pokémon Wiki. *Estadísticas*. Página web. Consultada el 24 de agosto de 2026.
- [4] Microsoft Learn. *Enumeraciones en C++*.
- [5] Microsoft Learn. *Constructores en C++*.
- [6] Microsoft Learn. *Uso de const en C++*.

- [7] Microsoft Learn. *Iteradores*.
- [8] Microsoft Learn. *Uso de `vector::push_back`*.
- [9] Microsoft Learn. *Control del formato de salida*.
- [10] *Diapositivas 02.2-Secuencias(3).pptx* y *03-Vectores y Listas(3).pptx*.
- [11] *Diapositiva 02.1-TAD(3).pptx sobre estado e interfaz de un TAD*.